

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INTERACTIVE TREE SIMULATOR

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO3 – Precision (BL3, BL5)	Assessment:	Assessment Tool 1 – Interactive Tree Simulator
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	<i>Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, B-Trees, Red-Black Trees, and Splay Trees.</i>		
Student Name:	G. Nithya Sree	Reg. No.:	192424103
Section / Batch:	3 rd Year AIDS	Date:	27/07/2026

Code of Conduct

I G. Nithya Sree (192424103) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Nithya Sree

Instructions

- This assessment tool contains 5 Interactive Tree Simulator questions, Total: 100 Marks.
- Students can use any online/offline Interactive Tree Simulator. Demonstrate insertion, deletion, searching, and traversal operations wherever applicable.
- When a student answer using simulator, in evaluation the answer should have captured screenshots after every major operation.
- Submit the report in PDF format with properly labelled screenshots as proof of completion.

Section / Topic	Focus	Q Nos	Marks
Binary Search Tree	Properties, binary tree and binary search tree, insertion and deletion	1	20
Tree Traversals	Traversal types, In order, Post order and Pre order	2	20
AVL Tree	Balancing factor, Rotation, Types of rotation, examples	3	20
B Tree, 2-3 tree, 2-3-4 tree	Properties, structures and Operations	4	20
Red Black Tree	Properties, Example and Operations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Interactive Tree Simulator

- Construct a Binary Search Tree (BST) by inserting the following keys: 40, 20, 10, 30, 60, 50, 70
Perform:
 - Inorder Traversal
 - Search for 70
 - Delete 50
 - Display the final BST.
- Complete Tree Traversals by Inserting 55,35,75,25,45,65,85. Find all the traversals.
 - Inorder
 - Preorder
 - Postorder
- Insert 40, 20, 60, 10, 30, 50, 70, 5 into an AVL Tree. Determine the balance factor of every node before and after balancing.
- Construct a B-Tree of order 3 by inserting 10, 20, 30, 40, 50.
- Construct a Red-Black Tree using 50, 40, 60, 30, 45, 55, 70, 20. Delete 30 and analyze how Red-Black properties are restored.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Correct Tree Construction	20	Tree is constructed correctly with all operations performed accurately	Minor errors in construction	Some operations incorrect	Tree construction mostly incorrect
Understanding of Tree Operations	20	Clearly explains insertion, deletion, search and balancing operations	Explains most operations correctly	Limited understanding	Unable to explain operations
Analysis of Tree Behaviour	30	Correctly analyses balancing, rotations, node split/merge, splaying	Good analysis with minor omissions	Partial analysis	Poor or incorrect analysis
Presentation	20	Well-organized report with accurate observations and conclusion	Good report with minor issues	Basic report with limited observations	Incomplete report
Accuracy of Responses	10	90–100% correct answers	75–89% correct answers	50–74% correct answers	Below 50% correct answers

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

QUESTION-1:

SEE ALGORITHMS

ArticlesAboutSign in

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

Red-Black Tree

AVL Tree vs Red-Black Tree

```
else:
    insert(node.right, key)
```

Enter a number: 59

INSERTDELETECLEAR

```
graph TD
    40((40)) --> 20((20))
    40 --> 60((60))
    20 --> 10((10))
    20 --> 30((30))
    60 --> 50((50))
    60 --> 70((70))
```

Controls

Input

Input is level-by-level order.

40,20,10,30,60,50,70

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

PreInPost

Generate Default

Visualize

Start Simulation

30206040501070

```
graph TD
    40((40)) --> 20((20))
    40 --> 10((10))
    20 --> 30((30))
    20 --> 60((60))
    10 --> 50((50))
    10 --> 70((70))
```

Enter a number: 46

INSERTDELETECLEAR

```
graph TD
    40((40)) --> 20((20))
    40 --> 60((60))
    20 --> 10((10))
    20 --> 30((30))
    60 --> 70((70))
```

QUESTION-2

← Back to Home

Tree Traversals

?

Controls

Input

Input is level-by-level order.

55, 35, 75, 25, 45, 65, 85

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

PreInPost

Generate Default

Visualize

Start Simulation

25354555657585

55

35

75

25

45

65

85

← Back to Home

Tree Traversals

?

Controls

Input

Input is level-by-level order.

55, 35, 75, 25, 45, 65, 85

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

PreInPost

Generate Default

Visualize

Start Simulation

55352545756585

55

35

75

25

45

65

85

← Back to Home

Tree Traversals

?

Controls

Input

Input is level-by-level order.

55, 35, 75, 25, 45, 65, 85

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

PreInPost

Generate Default

Visualize

Start Simulation

25453565857555

55

35

75

25

45

65

85

QUESTION-3

 SEE ALGORITHMS

Articles  Ab

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

rebalance(node.parent)

Enter a number: 21

INSERT

DELETE

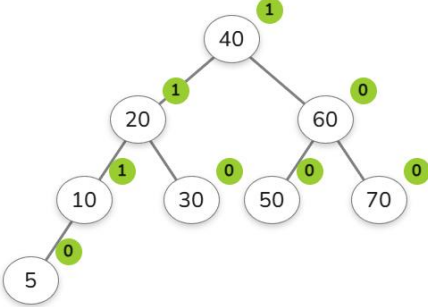
CLEAR

↶

↷

📁

🔗



The diagram shows an AVL tree with root 40 (balance factor 1). Node 40 has left child 20 (balance factor 1) and right child 60 (balance factor 0). Node 20 has left child 10 (balance factor 1) and right child 30 (balance factor 0). Node 10 has left child 5 (balance factor 0). Node 60 has left child 50 (balance factor 0) and right child 70 (balance factor 0).

QUESTION-4

 SEE ALGORITHMS

Articles  About [Sign in](#)

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

or keys, it **splits** — the median key is pushed up to the parent, while the remaining keys form two child nodes. This process can propagate upward and may even create a new root. This splitting mechanism is essential for keeping the tree balanced, ensuring that all leaves always remain at the same depth as keys are redistributed.

Enter a number: 53

INSERT

SEARCH

CLEAR

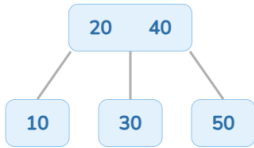
↶

↷

🔄

📁

🔗



The diagram shows a B+ tree with root node [20 | 40] and three leaf nodes [10], [30], and [50].

QUESTION-5

SEE ALGORITHMS

[Articles](#) [About](#)

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

Red-Black Tree

leaves (null nodes) are black, **red nodes cannot have red children**, and every path from a node to its descendant leaves contains the same number of black nodes.

recoloring to restore properties.

4. Repeat until all rules are satisfied from the modification to the root.

Enter a number: 21



INSERT

CLEAR

